2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD) | 979-8-3315-1560-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCAD66269.2025.11240916

Adder-DCIM: A Parallel Bit-Flexible Digital CIM Accelerator Joint Model Compression Framework for AdderNet Inference

Haikang Diao, Chuyue Tang, Bocheng Xu, Haoyang Luo, Meng Li, Yuan Wang, Xiyuan Tang
Peking University, Beijing, China. Email: diaohaikang@stu.pku.edu.cn, xitang@pku.edu.cn

***Abstract*—Heavily constrained edge-side tasks necessitate AI chips with low power consumption, low latency, and low cost. In recent years, digital compute-in-memory (CIM) has emerged as a promising solution to enhance energy efficiency and throughput density. However, digital CIM still faces various challenges: significant power and area overhead from multiplication, difficulty in exploiting fine-grained sparsity, and throughput degradation associated with bit-serial architecture. In this work, we propose Adder-DCIM: an efficient parallel bit-flexible DCIM accelerator joint model compression framework for AdderNet inference, in which the key contributions are: 1) a CIM-friendly model compression framework that includes operator decomposition, lossless fine-grained sparsity, and Kullback-Leibler-divergence(KLD)-based C_{in}-wise mixed-precision quantization; 2) a synchronous parallel DCIM architecture for throughput improvement with mix-precision quantization; 3) a bit-flexible minimal selector circuit for efficient mixed-precision computation. The experimental results demonstrate that under a 28-nm process, the proposed Adder-DCIM achieves a peak energy efficiency of 134 TOPS/W and a peak throughput density of 6.49 TOPS/mm² at INT8. When running ResNet20 on CIFAR10 and ResNet50 on ImageNet, the proposed Adder-DCIM achieves 255 TOPS/W@4.2bit and 243 TOPS/W@4.4bit with only a slight decrease in accuracy by 0.34% and 0.7%, respectively. Compared to multiply-based DCIM, Adder-DCIM improves energy efficiency × throughput density metrics by 20.7× for ResNet50 inference.**

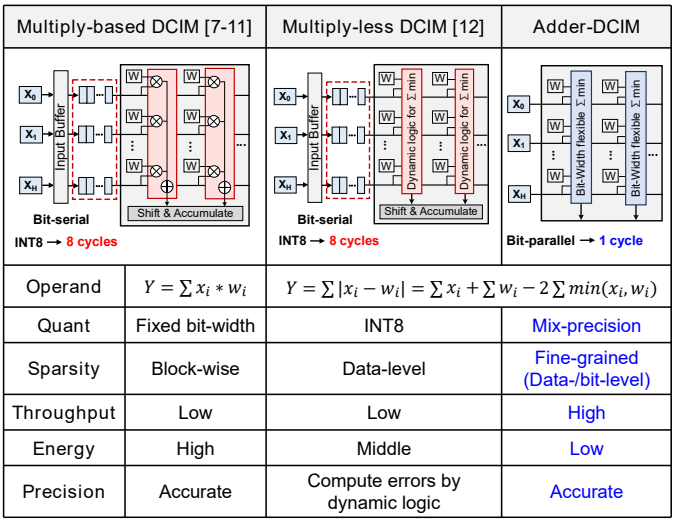
***Index Terms*—compute-in-memory (CIM), AdderNet, bit-flexible, mix-precision quantization.**

I. Introduction

Artificial intelligence (AI) based on neural networks (NN) has been widely applied in edge-side scenarios such as face recognition and voice interaction [1]–[4]. Compared to cloud computing, edge intelligence faces challenges like limited power supply and cost constraints, necessitating AI chips with low power consumption, low cost, and low latency. Due to the large number of parameters in NN, traditional AI chips based on the von Neumann architecture encounter the "memory wall" problem, where excessive data movement severely limits performance, making them unsuitable for edge intelligence tasks [5], [6].

In recent years, digital compute-in-memory (DCIM) architecture has garnered widespread attention [7]–[11]. This architecture significantly reduces data movement by integrating computing units into memory arrays and activating multiple rows in parallel for multiply-accumulate (MAC) operations, thereby improving the efficiency and performance of edge intelligence. Typically, DCIM architecture is combined with integer quantization algorithms for NN models, performing integer MAC computations within the CIM macro. Compared to the FP32 data type, quantizing data to INT8 improves the efficiency of DCIM while maintaining inference accuracy. However, as shown in Fig. 1, the current DCIM architecture still faces several challenges. The area and power overhead associated with multiplication grows quadratically with bit-width, leading to a significantly decreased area and energy efficiency. To mitigate the area overhead of multiplication, DCIM often employs a bit-serial dataflow, in which only one bit of the input is processed by the CIM macro per cycle. While this approach reduces area costs, it also substantially decreases throughput. For example, 8 cycles are required to complete the INT8 MAC computation, resulting in an 8× reduction in throughput. Furthermore, commonly used fine-grained sparsity algorithms are difficult to implement in the crossbar-structured DCIM. As a result, coarse-grained sparsity, such as block-wise sparsity, is more commonly adopted for DCIM, resulting in larger accuracy loss than fine-grained sparsity.

To address the high overhead associated with multiplication, a multiply-less approximate DCIM based on AdderNet has



	Multiply-based DCIM [7-11]	Multiply-less DCIM [12]	Adder-DCIM
Operand	$Y = \sum x_i * w_i$	$Y = \sum \lvert x_i - w_i \rvert = \sum x_i + \sum w_i - 2\sum min(x_i, w_i)$	
Quant	Fixed bit-width	INT8	Mix-precision
Sparsity	Block-wise	Data-level	Fine-grained (Data-/bit-level)
Throughput	Low	Low	High
Energy	High	Middle	Low
Precision	Accurate	Compute errors by dynamic logic	Accurate

Fig. 1: Comparisons of three types of DCIM works.

This work was supported in part by the Beijing Natural Science Foundation (L244051) and the Beijing Major Science and Technology Project (No. Z241100004224004). Corresponding author: Xiyuan Tang.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:27 UTC from IEEE Xplore. Restrictions apply.

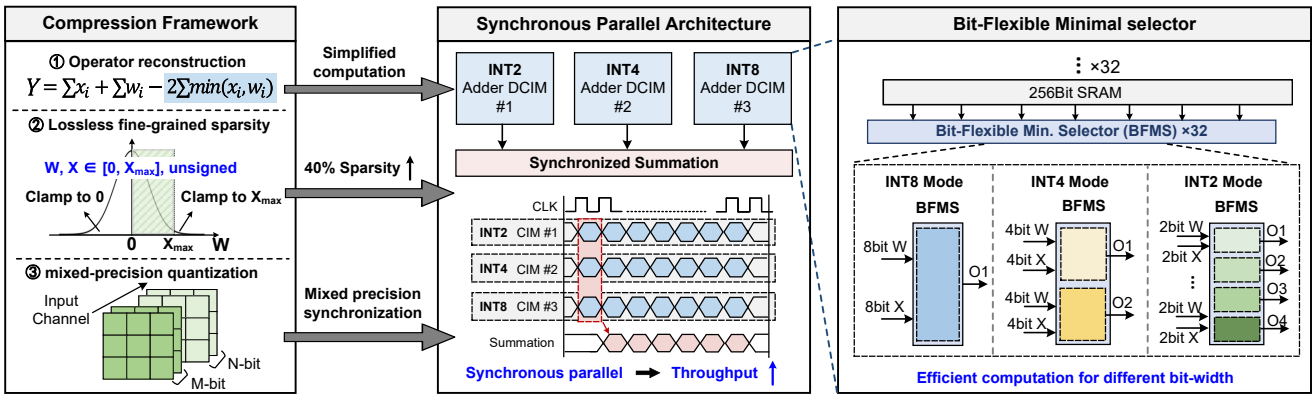


Fig. 2: The algorithm-architecture-circuit co-design for Adder-DCIM

been proposed [12]. AdderNet replaces the traditional dot product between weights and activations with L1 distance, significantly reducing computational complexity [13]. However, as shown in Fig. 1, multiply-less approximate DCIM remains some challenges. It only implements INT8 quantization for AdderNet without extending to lower bits, limiting its applicability in edge-side tasks that have extreme power constraints. The multiply-less approximate DCIM expands the L1-distance formula and transforms the bit-parallel absolute-difference operation into the bit-serial minimum selection operation, which also introducing significant throughput degradation. Additionally, the dynamic logic-based approximate comparator circuits in CIM macro also suffer from the issue of charge sharing, leading to random errors in comparison results and higher power consumption compared to static CMOS circuits.

In this work, we propose Adder-DCIM, a parallel bit-flexible DCIM for AdderNet inference. As shown in Fig. 2, it is a joint design of a CIM-friendly model compression framework, a synchronous parallel architecture, and a bit-flexible minimal selector circuit for efficient mixed-precision inference. This work employs co-optimization from algorithmic to circuit level, significantly enhancing energy efficiency and throughput density to meet the demands of low power, low latency, and low cost in edge-side applications. The primary contributions of this work are listed as follows:

- At the algorithm level, we propose a CIM-friendly model compression framework for AdderNet. It consists of operator decomposition, lossless fine-grained sparsity, and Kullback-Leibler divergence (KLD) based C_{in}-wise mixed-precision quantization to achieve extreme model compression.
- At the architectural level, we propose a synchronous parallel DCIM accelerator architecture for throughput improvement with mix-precision quantization. Minimal selection operations across different bit-widths are completed in a single cycle, facilitating synchronous C_{in} summation of multiple CIM Macros and achieving 100% hardware utilization.
- At the circuit level, we propose a bit-flexible minimal selector for parallel minimal selection operation at different bit-widths. This selector supports various bit-width operations using a multi-level comparison structure. Furthermore, it employs a fully static-based design, effectively leveraging bit-level sparsity.

TABLE I: The Performance Comparison.

Task	Model		Dataset	Performance	
				Multiply-based	AdderNet
Image Classification	CNN [13]	ResNet-20	CIFAR-100	68.1%	67.6%
		ResNet-50	ImageNet	76.2%	74.9%
	Transformer [15]	DeiT-B		81.8%	80.4%
		DeiT-S		79.9%	78.3%
Object Detection	R-CNN [16]		COCO	37.9% (mAP)	37.0% (mAP)
Super Resolution	VDSR [17]		B100	31.9 (PSNR)	31.8 (PSNR)
Image Compression	NTCF [18]		Kodak	33.8 (PSNR)	31.9 (PSNR)

The rest of the paper is organized as follows. Section II explains the background; Section III describes the model compression framework; Section IV describes the DCIM accelerator design; Section V shows the experimental results; Section VI concludes the paper.

II. Preliminaries

A. Multiply-less NN Algorithm and Quantization

To reduce the high overhead of multiplication computation, recent algorithms have explored other computational paradigms instead of multiplication. For example, AdderNet uses L1 distances instead of dot-product between weights and inputs [13]:

$$Y(m,n,t) = -\sum_{i=0}^{d}\sum_{j=0}^{d}\sum_{k=0}^{c_{in}}|X(m+i,n+j,k) - W(i,j,k,t)| \quad (1)$$

where d denotes the kernel size and C_{in} denotes the number of input channels. Compared to multiplication, the L1 distances operation in AdderNet substantially simplifies the computational complexity and improves chip efficiency. A theoretical analysis of the AdderNet is conducted in [14] to prove that it has the same expressive ability as the multiplication-based NN. In addition, Table I shows the performance of the AdderNet

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:27 UTC from IEEE Xplore. Restrictions apply.

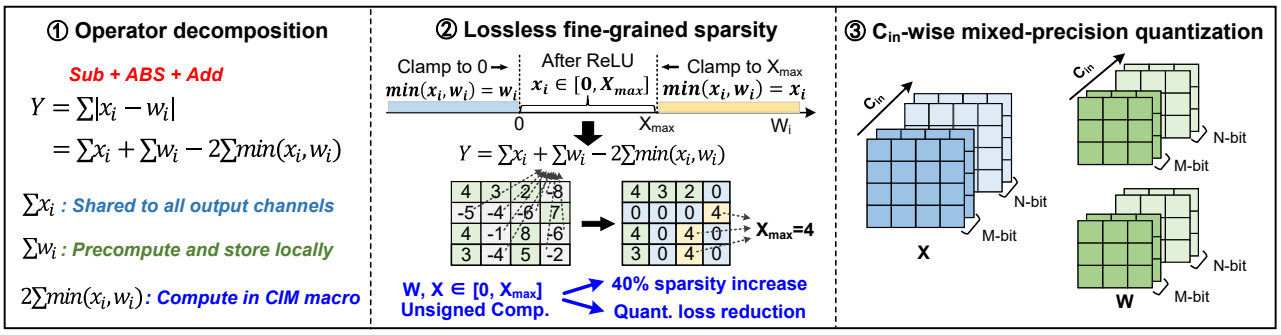


Fig. 3: Model compression framework for AdderNet

across different models (e.g., CNN and Transformer) and different tasks (e.g., object detection and image super-resolution) [13], [15]–[18], indicating that it can achieve comparable performance and scalability to multiply-based NN.

Low-bit quantization is a commonly used approach to further improve computational efficiency. Since the multiplicative union law cannot be applied to the kernel of AdderNet, the independent scales s_w and s_x cannot be adopted when quantifying weights and activations:

$$|X - W| = |(s_x X_q) - (s_w W_q)| \neq (s_x s_w) * |X_q - W_q| \quad (2)$$

Therefore, the existing works on AdderNet quantization adopt one shared scale s to quantize the weights and activations:

$$|X - W| = |(sX_q) - (sW_q)| = s * |X_q - W_q| \quad (3)$$

where $s = 2*\max(|W|)/(2^b-1)$ or $s = 2*\max(|X|)/(2^b-1)$ [19]. However, the data range of weights in AdderNet often far exceeds the activations [20]. Consequently, when weights and activations are quantified using a shared scale, significant quantization loss occurs regardless of whether the scale is determined from weights or activations. This loss leads to notable accuracy degradation, particularly in low-bit scenarios.

Recent studies have focused on optimizing the quantization process in AdderNet. For example, a weight standardization technique that facilitates weight quantization (WSQ) during model training is proposed in [21]. This approach reduces distribution discrepancies between weights and activations while achieving 89.9% inference accuracy with INT8 precision on CIFAR-10. However, thorough evaluation on more challenging benchmarks like ImageNet remains unexplored. Moreover, an activation-oriented (AOO) quantization scheme is proposed in [22] to achieve low quantization bit-width. However, this method incurs accuracy degradation of 1.7% at INT6 and 15% at INT4 compared to the FP32 baseline. To address these limitations, we propose a lossless weight sparsity method that aligns the distribution ranges of weights and activations, enabling simultaneous improvement in quantization efficiency (reduced bit-width) and inference accuracy.

B. Multiply-less DCIM Design

Based on AdderNet, multiply-less approximation DCIM is proposed in [12] to eliminate the overhead of multiplication computation circuits. As shown in Fig 1, the multiply-less DCIM expands the L1-distance formula and transforms the bit-parallel absolute-difference operation into the bit-serial minimum selection operation. Then the multiply-less DCIM employs a dynamic logic-based approximate comparator to further reduce the circuit's area and energy. However, this dynamic logic-based comparator suffers from the charge-sharing problem, which leads to computational errors occurring. Therefore, the multiply-less approximation DCIM requires at least INT8 precision to avoid significant degradation of inference accuracy.

III. CIM-Friendly Model Compression Framework

To reduce the overhead of AdderNet computation in DCIM, we propose a DCIM-friendly model compression framework. As illustrated in Fig. 3, the model compression flow is divided into 3 steps: operator decomposition, lossless fine-grained sparsification, and C_{in}-wise mixed-precision quantization. It should be noted that many model compression methods are incompatible with DCIM architectures. For example, conventional fine-grained sparsity algorithms cannot effectively deploy only dense weights on crossbar-structured DCIMs, thereby failing to enhance actual hardware efficiency. The proposed model compression framework in this work is specifically designed to be adaptable to DCIM architecture, allowing for circuit simplification with little accuracy loss.

The L1 distance operation of AdderNet contains three steps of subtraction, taking absolute values, and summation, leading to high computational latency. To fully leverage the CIM architecture, we expand and reconstruct the operator similarly to [12], transforming the L1 distance into a minimal summation. Since different columns in the CIM are deployed with weights for different output channels, $(\sum x_i)$ is computed once and then shared across all the output channels to amortize the extra computation costs. $(\sum w_i)$ is precomputed during model compilation and stored locally in the CIM. Last, only the minimal summation, $2\sum min(x_i, w_i)$, needs to be computed in the CIM macro. This operator reconstruction reduces both the computational delay and complexity compared to the original L1 distance computation.

As shown in Fig. 4(a)(b), the range of weights is larger than that of activations in the AdderNet [20]. Therefore, we can leverage this data distribution characteristic to pre-determine

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:27 UTC from IEEE Xplore. Restrictions apply.

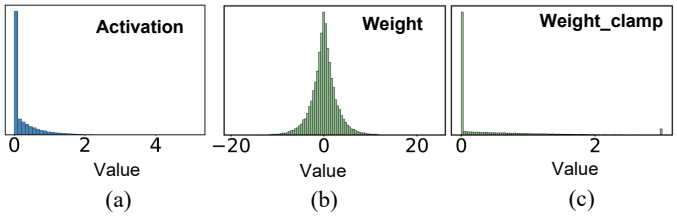


Fig. 4: (a) Distribution range of activation after ReLU function, (b) distribution range of weights before clamping, and (c) distribution range of weights after clamping.

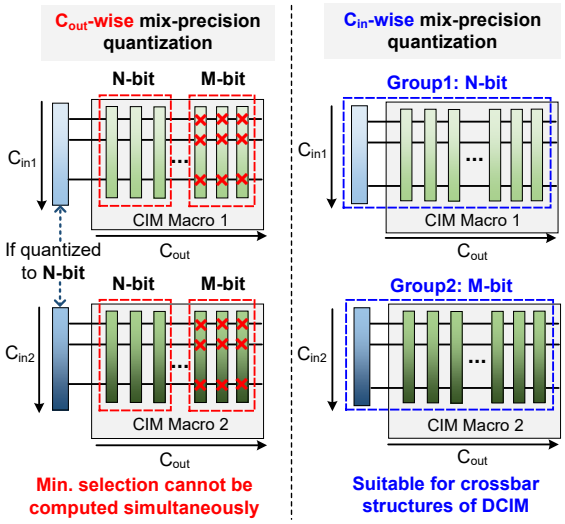


Fig. 5: Comparison of mixed-precision quantization for grouping along input channels (C_{in}) and output channels (C_{out})

certain results to realize lossless fine-grained sparsity as shown in the middle of Fig. 3. After the ReLU activation function, the activation X is clamped to [0, X_{max}], where X_{max} is an empirical value obtained from the training set. For weight less than 0, as the X is non-negative, the minimum value can be directly determined to be the weight itself, without the need for selection. Therefore, negative weights can be directly clamped to 0, and combined with $\sum w_i$ in advance. Similarly, for weight greater than X_{max}, it can be pre-determined that the minimal is X_{max}, allowing these weights to be clamped to X_{max}. By employing lossless clamping, weights are clamped to [0, X_{max}] as shown in Fig. 4(c), achieving over 40% fine-grained sparsity increase without accuracy loss. Since all activations and weights are non-negative, the result of the minimal summation must also be non-negative. Therefore CIM only needs to support unsigned computation, simplifying the circuit design. In addition, the weight distribution range is aligned with the activations, which can substantially reduce the quantization loss caused by the shared scale.

Mixed-precision quantization maintains inference accuracy while efficiently reducing the average quantization bit-width. It groups activations and weights based on importance or numerical distribution characteristics, assigning differentiated bit-width for quantization in different groups. Current mixed-precision quantization methods primarily include layer-wise

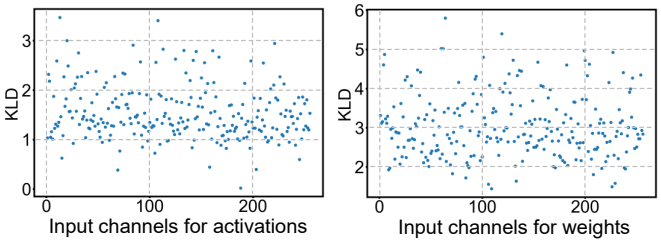


Fig. 6: KLD for all input channels at layer 10 of the ResNet50.

quantization [23], [24] and output-channel-wise (C_{out}-wise) quantization [25]. The latter generally achieves a lower average bit width through more fine-grained quantization. However, in this work, the mixed precision quantization in C_{out} is not suitable for AdderNet-based CIM in this work. As shown on the left of Fig. 5, if the weights on different output channels have a varying bit width, 1 activation cannot simultaneously perform the minimal selection operation with weights of different bit widths. To address this issue, we propose mixed precision quantization in C_{in}-wise, as shown on the right of Fig. 5. In this method, different input channels are grouped, and each group's activations and weights are quantized to the same bit width, thereby enabling parallel minimal selection operations within the CIM macro.

Before grouping input channels, we adopt Kullback-Leibler divergence (KLD) to calculate the quantization loss of weights and inputs for different input channels in each layer. KLD is a common metric for measuring the difference between two probability distributions and is used in the TensorRT quantization algorithm [26]:

$$K = \sum P(X) \log \frac{P(X)}{Q(X)} \tag{4}$$

where $P(X)$ represents the true probability distribution of the original data, and $Q(X)$ represents the probability distribution of quantized data. A higher KLD value indicates a larger difference between the distribution of the original FP32 data and the quantized data, implying a larger quantization loss. Fig. 6 presents the KLD values of all input channels in layer 10 of ResNet50. Even within the same layer, the quantization losses of different input channels exhibit large differences. Based on this, input channels with lower KLDs are assigned lower bit widths, while those with higher KLDs are assigned higher bit widths.

The specific mixed-precision quantization flow is shown in Algorithm 1. After operator reconstruction and weight clamping, we perform INT4 quantization to obtain the quantized X_{Quant} and W_{Quant}. Next, for each input channel of every layer, the KLD of activations and weights is calculated separately. Since the weights and activations in AdderNet need to adopt the same bit width, we calculate the weighted average of the KLD of the activations (X_{KLD}) and weights (W_{KLD}) to obtain the final KLD value ($K^{L\times C_{in}}$). Then, we sort and group the input channels with similar KLD values within each layer to obtain the KLD of each group, denoted

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:27 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 : Model Compression Flow for AdderNet

Input: FP32 AdderNet model; Number of layers L; Number of channels per group G_{size}; Number of groups per layer G_{num}.

Output: Mix-precision quantized model;

1: Operator decomposition
2: NN Forward to find each layer's X_{max}
3: Weight clamping
4: INT4 quantization to obtain the X_{Quant} and W_{Quant}
5: **for** $i = 1 : L$ **do**
6: **for** $j = 1 : C_{in}$ **do**
7: $X^{i,j}_{KLD} = \sum P(X^{i,j}) \log \frac{P(X^{i,j})}{P(X^{i,j}_{Quant})}$
8: $W^{i,j}_{KLD} = \sum P(W^{i,j}) \log \frac{P(W^{i,j})}{P(W^{i,j}_{Quant})}$
9: $K^{i,j} = \frac{X^{i,j}_{KLD} * Size(X^{i,j}) + W^{i,j}_{KLD} * Size(W^{i,j})}{Size(X^{i,j}) + Size(W^{i,j})}$
 where P(*) is the probability distribution, Size(*) is the number of the parameter
10: **end for**
11: **end for**
12: KLD sorting and C_{in} grouping for each layer by $K^{L \times C_{in}}$
13: KLD for each group computation: $KG^{L \times G_{num}} = \sum_{j=0}^{G_{size}} K^{i,j}$
14: NN Forward to find each group's maximum: XG_{max}
15: Group-wise weight clamping: $W_c \in [0, XG_{max}]$
16: bit width allocation and quantization according to $KG^{L \times G_{num}}$

as $KG^{L \times G_{num}}$ (lines 12-13 in Algorithm 1). Finally, after the weights clamping for each group, we assign different bit-width to achieve mixed-precision quantization based on $KG^{L \times G_{num}}$.

IV. Parallel Bit-Flexible DCIM Accelerator

Based on the model compression framework, we design a parallel bit-flexible DCIM accelerator as shown in Fig. 7. The DCIM accelerator contains 64KB of global memory, 32KB of ACT buffer, 12 DCIM macros, a summation module, and a SIMD core. Each CIM macro contains 32 columns. Each column contains 256 SRAM bit cells, 32 bit-flexible minimal selectors (BFMS), and 1 adder tree to perform minimum selection and summation operations. The summation module collects $\sum x_i$, $\sum w_i$, and $min(x_i, w_i)$) to generate partial sums (PSUM), and subsequently sums the PSUM values across input channel dimensions to produce the final output. The SIMD core handles operations such as ReLU, de-quantization, and max-pooling.

A. Bit-Flexible Minimal Selector

Fig. 8(a) shows the circuitry of the BFMS. It contains 8 encoders, a 3-level comparison tree, 4 multiplexers, and 8 selectors. A hard-wired design is employed in BFMS to avoid the power overhead associated with SRAM pre-charging. The Q nodes ($W_{0\sim7}$) and QB nodes ($WB_{0\sim7}$) of 8 SRAM bit-cells are directly connected to the BFMS. To reduce the latency of combinational logic, a carry-lookahead design is adopted. The

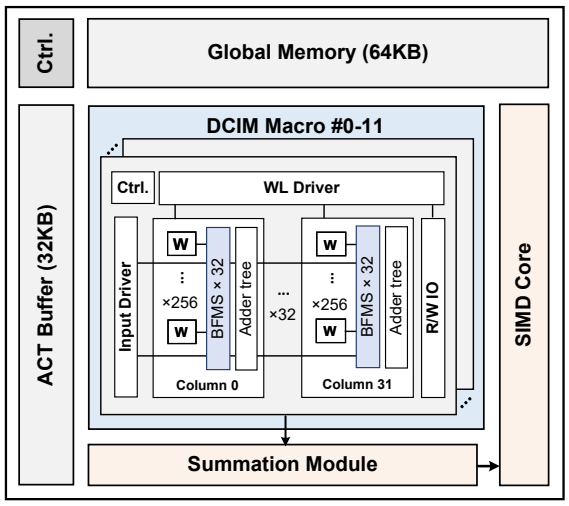


Fig. 7: Overall architecture of Adder-DCIM.

8-bit signals $XB_{0\sim7}$ (the inverse of X), $W_{0\sim7}$, and $WB_{0\sim7}$ are converted into 8-bit generate signals ($G_{0\sim7}$) and 8-bit propagate signals ($P_{0\sim7}$):

$$\begin{aligned} G_i &= W_i \ \& \ XB_i \\ P_i &= WB_i \ \bigoplus \ XB_i \end{aligned} \tag{5}$$

where, i = 0∼7. The 3-level comparison tree calculates the comparison results where 3 levels output the comparison results for INT2, INT4, and INT8 modes respectively. Specifically, the first level outputs the comparison result of INT2:

$$L2_{[2i+1;2i]} = \overline{G_{2i+1} + P_{2i+1} \& G_{2i}} \tag{6}$$

where, i = 0, 1, 2, 3. The second level outputs the comparison result of INT4:

$$\begin{aligned} L4_{[3:0]} &= L2_{[3:2]} + P_{[3:2]} \& L2_{[1:0]} \\ L4_{[7:4]} &= L2_{[7:6]} + P_{[7:6]} \& L2_{[5:4]} \end{aligned} \tag{7}$$

where $P_{[3:2]} = \overline{P_3 \& P_2}, P_{[7:6]} = \overline{P_7 \& P_6}$. The third level outputs the comparison result of INT8:

$$L8 = \overline{L4_{[7:4]} + P_{[7:4]} \& L4_{[3:0]}} \tag{8}$$

where $P_{[7:4]} = P_7 \& P_6 \& P_5 \& P_4$. Then, 4 multiplexers are configured based on bit-width to select the appropriate comparison results, which are then sent to 8 selectors. Each selector contains two transmission gates and a tri-state inverter controlled by EN and ENB signals, reducing the adder tree's dynamic power caused by unnecessary flipping from the BFMS. As shown in Fig. 8(c), the EN and ENB signals are employed to gate the BFMS output at the rising edge of the clock, and after a delay, transmit the BFMS output to the adder tree. Through the tri-state inverter, the power consumption of the adder tree can be reduced by 44%. Once the BFMS completes computation, settings EN=1 and ENB=0 and transfers the $min(x_i, w_i)$ to the adder tree. Additionally, we designed a configurable delay generation model in the chip that can adjust the delay time to accommodate different operating frequencies and voltages. Due to the low combinational logic delays of

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:27 UTC from IEEE Xplore. Restrictions apply.

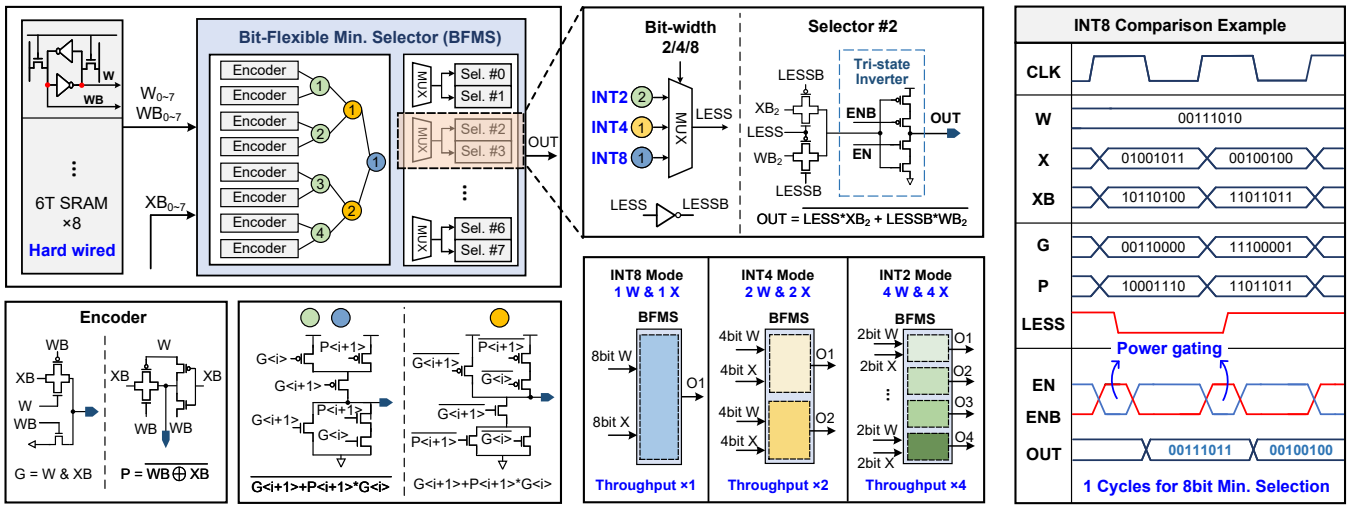


Fig. 8: (a) The detail circuits of the bit-flexible minimal selector (BFMS), (b) three operation modes for INT8/4/2 computation respectively, and (c) the waveform for BFMS.

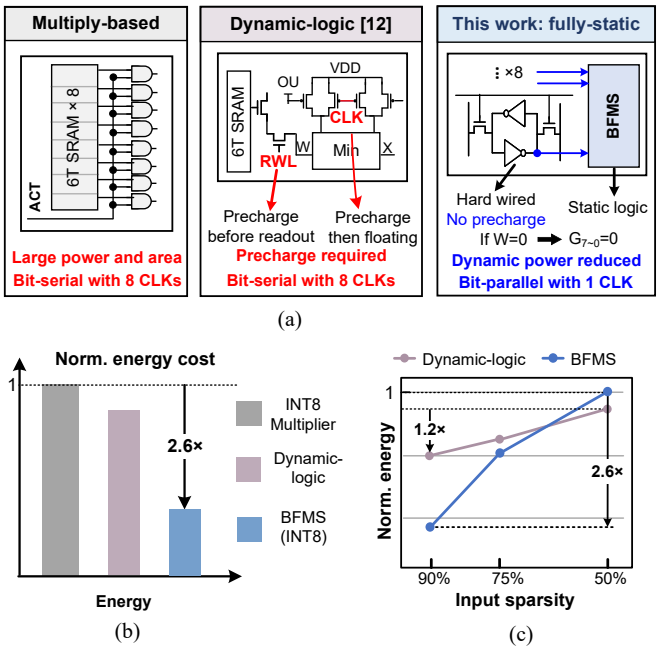


Fig. 9: (a) Three digital computing circuit designs for DCIM, (b) normalized energy when normalized to the same throughput, and (c) normalized energy at different bit-level sparsity.

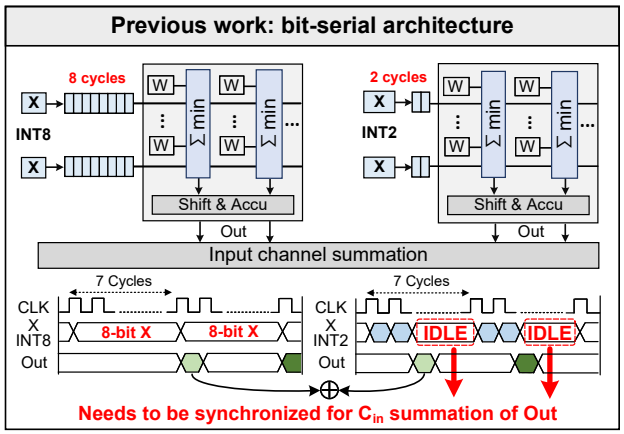
the BFMS, the OUT node is floated for less than 1/8 clock cycle. Therefore, the voltage drop at the OUT node is minimal and does not increase the power consumption of the back-end circuit.

Fig. 8 (b) illustrates the three operation modes of the BFMS for INT8/4/2, respectively. Based on the 3-level comparison tree design, the BFMS flexibly supports computations with different bit-width. In INT4 mode, the BFMS can simultaneously perform two min-select operations, which improves the throughput by 2× compared to INT8. In INT2 mode, the BFMS can simultaneously perform four min-select operations, which improves the throughput by 4×.

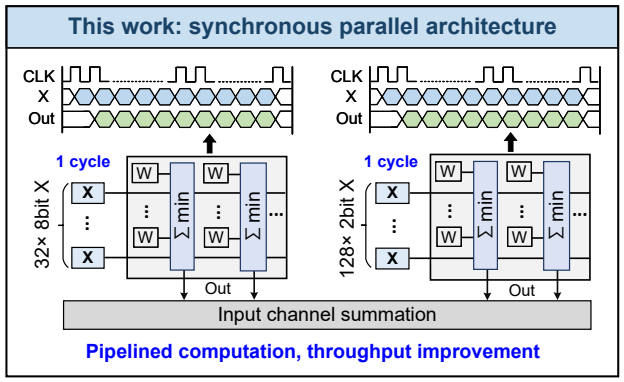
Fig. 9(a) illustrates three digital computing circuit designs for DCIM, including the dynamic logic design previously employed for AdderNet inference [12]. While the first two conventional architectures adopt bit-serial computation to minimize area overhead, they require 8 cycles to complete an INT8 precision operation. In contrast, our proposed bit-flexible parallel comparator (the right of Fig. 9(a)) achieves single-cycle completion through architectural innovation, thereby improving computational throughput by 8×. Fig 9(b) quantifies the normalization cost advantages of BFMS. Through the elimination of multiplication and custom circuit design, BFMS demonstrates 2.6× lower power compared to conventional INT8 multipliers, dramatically reducing the overhead of digital logic circuits in the DCIM.

In addition, as mentioned in Section III, all negative weights in AdderNet are clamped to 0, substantially increasing the sparsity of the weights. However, previous multiply-less DCIM is difficult to exploit bit-level sparsity [12]. As shown in the middle of Fig. 9(a), it adopts 8T SRAM and dynamic logic-based comparators to drastically reduce the area overhead of the digital logic circuits. However, both 8T SRAM and dynamic logic-based comparators need to be pre-charged before computation, regardless of whether the input bits are ”0” or ”1”. Therefore, dynamic logic-based design cannot reduce power consumption as sparsity increases. This work proposes a fully static-based design as shown in the right of Fig. 9(a). The BFMS is directly connected to the Q node and QB node of the SRAM via metal wires, thereby avoiding pre-charging. Additionally, the BFMS is based on static logic circuits and does not require pre-charging. For example, when the weights are 0, $G_{0\sim7}$ also becomes 0. In this case, most of the circuits in the BFMS no longer flip regardless of how X varies, greatly reducing the dynamic power. As shown in Fig. 9(c), the power

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:27 UTC from IEEE Xplore. Restrictions apply.



(a)



(b)

Fig. 10: (a) Bit-serial DCIM architecture and (b) parallel DCIM architecture for mix-precision computation.

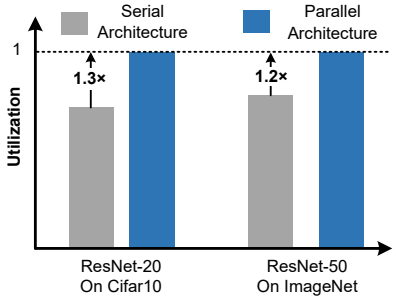


Fig. 11: Hardware utilization comparison between bit-serial architecture and synchronous parallel architecture.

consumption of BFMS under high bit-level sparsity is much lower than that of the dynamic logic-based minimal selector.

B. Synchronous Parallel DCIM Architecture

Previous DCIM works have typically employed a bit-serial architecture to minimize digital logic area within CIM macros. However, this architecture fundamentally conflicts with mix-precision quantization, as demonstrated in Fig. 10(a). Specifically, CIM macro-1 performs 8-bit computation and requires 8 cycles to output PSUM, while CIM macro-2 performs 2-bit computation and only requires 2 cycles to output PSUM. This temporal mismatch forces CIM macro-2 to remain idle for 6 cycles during cross-macro synchronization for input channel summation, resulting in suboptimal 25% hardware utilization and severely degraded system throughput. In this work, we propose a synchronous parallel DCIM architecture as shown in Fig. 10(b). All CIM macros require only 1 cycle to output PSUM at different bit-width, enabling synchronous C_{in} summation across CIM macros and achieving 100% hardware utilization. As shown in Fig. 11, We evaluated on ResNet20 and ResNet50 models, respectively. Compared to the serial DCIM architecture, the synchronous parallel DCIM architecture improves hardware utilization by 1.3× and 1.2× for ResNet-20 and ResNet-50, respectively.

TABLE II: Quantization Results of AdderNet.

Method	ResNet-20 on Cifar10			ResNet-50 on ImageNet		
	ACT-bits	W-bits	Top-1	ACT-bits	W-bits	Top-1
Baseline [13]	32	32	91.84%	32	32	74.9%
WSQ [21]	8	8	89.9%	8	8	N/A
	4	4	<10%	4	4	N/A
AOO [22]	8	8	91.1%	8	8	76.1%
	4	4	<10%	4	4	<60%
QSSF [19]	4	4	87.57%	4	4	65.1%
REQ [20]	8	8	91.66%	8	8	74.7%
	4	4	90.98%	4	4	71.3%
Ours	4.2	4.2	91.50%	4.4	4.4	74.2%

V. Experiment Results

A. Evaluation of Mix-precision Quantization

We implement mixed-precision quantization for AdderNet and verify inference accuracy on the standard dataset. The quantization results are shown in the Table II. The quantization bit-width is limited to INT2, INT4, and INT8 to fit the hardware design. We also compare our approach with WSQ [21], AOQ [22], QSSF [19], and REQ [20]. WSQ-AdderNet and AOQ have been described in Section II-A. QSSF directly uses the shared quantization scale without truncation of weights, while REQ uses weight truncation and output channel grouping to reduce quantization loss. All of the methods quantize AdderNet with a fixed bit-width. However, as discussed in Section III, output-channel grouping is not suitable for AdderNet-based DCIM. Therefore, we removed output-channel grouping when replicating the REQ algorithm for a fair comparison.

For ResNet-20, 16 input channels are divided into groups and each group is configured with a different bit-width. With an average bit-width of 4.2, our method reduces Top-1 accuracy by only 0.34% compared to the FP32 benchmark. Compared to the INT4 results in REQ, our method can improve accuracy by 0.52%. For ResNet-50, 32 input channels are divided into groups. With an average bit-width of 4.4, the Top-1 accuracy is only reduced by 0.7% compared to the FP32 benchmark. Compared to the INT4 results in REQ, our method can improve accuracy by 2.9%.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:27 UTC from IEEE Xplore. Restrictions apply.

TABLE III: Comparison of Proposed Adder-DCIM with Previous Works.

	ISSCC'21 [7]	DAC'24 [27]	ISSCC'24[28]	ISSCC'23[12]	This work
Technology	22nm	28nm	28nm	28nm	28nm
CIM Macro Area	0.202mm²	1.96mm²	1.94mm²	0.028mm²	0.079mm²
Voltage	0.52-0.92V	0.6-0.9V	0.7-0.95V	0.54-0.9V	0.6-0.9V
Frequency	N/A	250MHz	118-209MHz	20-230MHz	50-250MHz
NN Operand	MAC	MAC	MAC	L1-distance	L1-distance
Bit-precision	INT8/12/16	INT4/8 Mix-precision	INT8	INT8	INT2/4/8 Mix-precision
Peak Energy Efficiency (TOPS/W)*	24.7@INT8 89@INT4	5.15@INT8 22.1@INT4 18.8@ResNet50	50.53@INT8	102@INT8	134@INT8 255@ResNet20 243@ResNet50
Peak Throughput Density (TOPS/mm²)*	4.5@INT8	N/A	2.74@INT8	4.4@INT8	6.49@INT8 12.4@ResNet20 11.8@ResNet50
Accuracy Loss for ResNet50	N/A	0.98% on Cifar100	0.05% on Cifar100	0.4% on ImageNet	0.7% on ImageNet

*We chose the macro-level results for a fair comparison. The energy efficiency and throughput density of our work contains ACT summation ($\sum x_i$), minimal summation, and the final integration ($\sum x_i - \sum w_i - 2\sum min(x_i, w_i)$).

B. Evaluation of DCIM Accelerator

To evaluate the proposed DCIM accelerator, we implemented a prototype system using industry 28-nm technology. The DCIM macro is designed by Virtuoso, and the area, power, and latency are estimated through post-layout simulation. The other digital modules use a digital flow design (digital synthesis by Design Compiler and digital backend by Innovus). The area, power, and latency are estimated using Innovus. Fig. 12 shows the layout of the prototype DCIM accelerator, which occupies a die area of 0.58 mm².

Table III provides a comprehensive comparison between the proposed Adder-DCIM and the state-of-the-art (SOTA) solutions [12], [27], [28]. The Adder-DCIM operates within a voltage range of 0.6-0.9V and a frequency range of 50-250 MHz. The peak macro energy efficiency for INT8 is 134 TOPS/W at 0.6V, 50MHz with 90% data sparsity, while the peak macro throughput density for INT8 is 6.49 TOPS/mm² at 0.9V, 250MHz. When running ResNet20 on CIFAR10 and ResNet50 on ImageNet, Adder-DCIM achieves 255 TOPS/W@4.2bit and 243 TOPS/W@4.4bit, with only slight accuracy reductions of 0.34% and 0.7%, respectively. Compared to multiply-less DCIM based on AdderNet [12], Adder-DCIM achieves energy efficiency and throughput density improvements of 2.4× and 2.7× for ResNet50 inference. Fig. 13 illustrates the improvements in efficiency and performance achieved by co-designing algorithms, architecture, and circuits. Compared to multiply-based CIM [28], Adder-DCIM improves energy efficiency × throughput density metrics by 20.7×, meeting the demand for low power and low latency in edge-side applications.

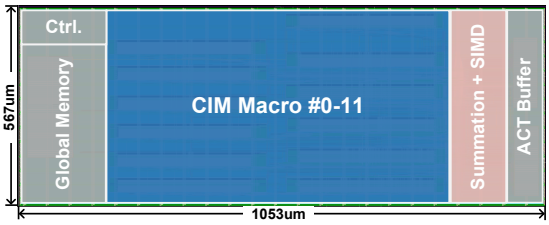


Fig. 12: The layout of the prototype DCIM accelerator.

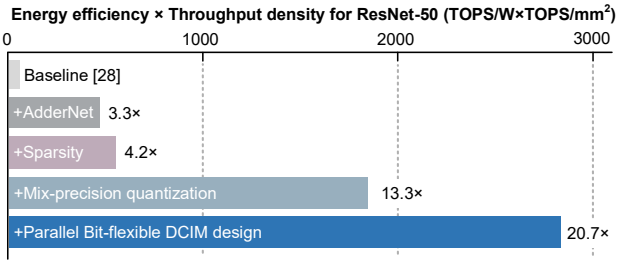


Fig. 13: Performance breakdown analysis.

VI. Conclusion

In this work, we propose Adder-DCIM: a parallel bit-flexible DCIM co-designed with the model compression framework for AdderNet inference. The proposed model compression framework includes operator decomposition, lossless fine-grained sparsity, and C_{in}-wise mixed-precision quantization based on Kullback-Leibler divergence (KLD). Meanwhile, a DCIM accelerator is proposed to implement mix-precision inference. It features a parallel architecture and a bit-flexible minimum selector that supports computations with varying bit-width, effectively eliminating hardware idleness caused by mixed-precision summation. Experimental results demonstrate that under a 28-nm process, Adder-DCIM achieves a peak macro energy efficiency of 134 TOPS/W and a throughput density of 6.49 TOPS/mm² at INT8. When running ResNet20 on CIFAR10 and ResNet50 on ImageNet, the proposed Adder-DCIM achieves 255 TOPS/W@4.2bit and 243 TOPS/W@4.4bit, with only slight accuracy reductions of 0.34% and 0.7%, respectively. Compared to previous AdderNet-based DCIM works, this work achieves 2.4× and 2.7× better in energy efficiency and throughput density, respectively.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:27 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, pp. 84 – 90, 2012.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6000–6010.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[5] N. P. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. Patterson, "Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," 2023.

[6] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun, and O. Temam, "Dadiannao: A machine-learning supercomputer," in *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*, 2014, pp. 609–622.

[7] Y.-D. Chih, P.-H. Lee, H. Fujiwara, Y.-C. Shih, C.-F. Lee, R. Naous, Y.-L. Chen, C.-P. Lo, C.-H. Lu, H. Mori, W.-C. Zhao, D. Sun, M. E. Sinangil, Y.-H. Chen, T.-L. Chou, K. Akarvardar, H.-J. Liao, Y. Wang, M.-F. Chang, and T.-Y. J. Chang, "16.4 an 89tops/w and 16.3tops/mm2 all-digital sram-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *2021 IEEE International Solid- State Circuits Conference (ISSCC)*, vol. 64, 2021, pp. 252–254.

[8] H. Mori, W.-C. Zhao, C.-E. Lee, C.-F. Lee, Y.-H. Hsu, C.-K. Chuang, T. Hashizume, H.-C. Tung, Y.-Y. Liu, S.-R. Wu, K. Akarvardar, T.-L. Chou, H. Fujiwara, Y. Wang, Y.-D. Chih, Y.-H. Chen, H.-J. Liao, and T.-Y. J. Chang, "A 4nm 6163-tops/w/b $\mathbf{4790-TOPS/mm^2/b}$ sram based digital-computing-in-memory macro supporting bit-width flexibility and simultaneous mac and weight update," in *2023 IEEE International Solid- State Circuits Conference (ISSCC)*, 2023, pp. 132–134.

[9] D. Wang, C.-T. Lin, G. K. Chen, P. Knag, R. K. Krishnamurthy, and M. Seok, "Dimc: 2219tops/w 2569f2/b digital in-memory computing macro in 28nm based on approximate arithmetic hardware," in *2022 IEEE International Solid- State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 266–268.

[10] J. Yue, C. He, Z. Wang, Z. Cong, Y. He, M. Zhou, W. Sun, X. Li, C. Dou, F. Zhang, H. Yang, Y. Liu, and M. Liu, "A 28nm 16.9-300tops/w computing-in-memory processor supporting floating-point nn inference/training with intensive-cim sparse-digital architecture," in *2023 IEEE International Solid- State Circuits Conference (ISSCC)*, 2023, pp. 1–3.

[11] B. Yan, J.-L. Hsu, P.-C. Yu, C.-C. Lee, Y. Zhang, W. Yue, G. Mei, Y. Yang, Y. Yang, H. Li, Y. Chen, and R. Huang, "A 1.041-mb/mm2 27.38-tops/w signed-int8 dynamic-logic-based adc-less sram compute-in-memory macro in 28nm with reconfigurable bitwise operation for ai and embedded applications," in *2022 IEEE International Solid- State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 188–190.

[12] Y. He, H. Diao, C. Tang, W. Jia, X. Tang, Y. Wang, J. Yue, X. Li, H. Yang, H. Jia, and Y. Liu, "7.3 a 28nm 38-to-102-tops/w 8b multiply-less approximate digital sram compute-in-memory macro for neural-network inference," in *2023 IEEE International Solid- State Circuits Conference (ISSCC)*, 2023, pp. 130–132.

[13] H. Chen, Y. Wang, C. Xu, B. Shi, C. Xu, Q. Tian, and C. Xu, "Addernet: Do we really need multiplications in deep learning?" in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 1465–1474.

[14] H. Chen, Y. Wang, C. Xu, C. Xu, C. Xu, and T. Zhang, "Universal adder neural networks," 2021.

[15] H. Shu, J. Wang, H. Chen, L. Li, Y. Yang, and Y. Wang, "Adder attention for vision transformer," in *2021 Advances in Neural Information Processing Systems(NIPS)*, vol. 34, 2021, pp. 19 899–19 909.

[16] X. Chen, C. Xu, M. Dong, C. XU, and Y. Wang, "An empirical study of adder neural networks for object detection," in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 6894–6905. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2021/file/37693cfc748049e45d87b8c7d8b9aacd-Paper.pdf

[17] D. Song, Y. Wang, H. Chen, C. Xu, C. Xu, and D. Tao, "Addersr: Towards energy efficient image super-resolution," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 15 643–15 652.

[18] B. Li, Y. Xin, C. Li, Y. Bao, F. Meng, and Y. Liang, "Adderic: Towards low computation cost image compression," in *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 2030–2034.

[19] Y. Wang, M. Huang, K. Han, H. Chen, W. Zhang, C. Xu, and D. Tao, "Addernet and its minimalist hardware design for energy-efficient artificial intelligence," 2021.

[20] Y. Nie, K. Han, H. Diao, C. Liu, E. Wu, and Y. Wang, "Redistribution of weights and activations for addernet quantization," in *2022 Advances in Neural Information Processing Systems(NIPS)*, vol. 35, 2022, pp. 22 739–22 751.

[21] Y. Zhang, B. Sun, W. Jiang, Y. Ha, M. Hu, and W. Zhao, "Wsq-addernet: Efficient weight standardization based quantized addernet fpga accelerator design with high-density int8 dsp-lut co-packing optimization," in *2022 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2022, pp. 1–9.

[22] Y. Zhang, O. A. Kailani, B. Zhou, and W. Zhao, "Addernet 2.0: Optimal addernet accelerator designs with activation-oriented quantization and fused bias removal-based memory optimization," *IEEE Transactions on Circuits and Systems I: Regular Papers*, pp. 1–11, 2025.

[23] Z. Dong, Z. Yao, A. Gholami, M. Mahoney, and K. Keutzer, "Hawq: Hessian aware quantization of neural networks with mixed-precision," 2019. [Online]. Available: https://arxiv.org/abs/1905.03696

[24] Z. Dong, Z. Yao, Y. Cai, D. Arfeen, A. Gholami, M. W. Mahoney, and K. Keutzer, "Hawq-v2: Hessian aware trace-weighted quantization of neural networks," 2019. [Online]. Available: https://arxiv.org/abs/1911.03852

[25] B. A. Motetti, M. Risso, A. Burrello, E. Macii, M. Poncino, and D. J. Pagliari, "Joint pruning and channel-wise mixed-precision quantization for efficient deep neural networks," *IEEE Transactions on Computers*, vol. 73, no. 11, pp. 2619–2633, 2024.

[26] S. Migacz, "8-bit inference with tensorrt," in *Proceedings of Conference on GPU Technology*, 2017, p. 7.

[27] B. Liu, Q. Wei, Y. Zhang, X. Xu, Z. Zou, X. Huang, X. Si, and H. Cai, "FDCA: fine-grained digital-cim based CNN accelerator with hybrid quantization and weight-stationary dataflow," in *Proceedings of the 61st ACM/IEEE Design Automation Conference, DAC 2024, San Francisco, CA, USA, June 23-27, 2024*, V. De, Ed. ACM, 2024, pp. 144:1–144:6. [Online]. Available: https://doi.org/10.1145/3649329.3656253

[28] Y. Yuan, Y. Yang, X. Wang, X. Li, C. Ma, Q. Chen, M. Tang, X. Wei, Z. Hou, J. Zhu, H. Wu, Q. Ren, G. Xing, P.-I. Mak, and F. Zhang, "34.6 a 28nm 72.12tflops/w hybrid-domain outer-product based floating-point sram computing-in-memory macro with logarithm bit-width residual adc," in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 67, 2024, pp. 576–578.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:27 UTC from IEEE Xplore. Restrictions apply.